



School of Future Tech

Case Study Report

STUDENT DETAILS

Student Name: Nishi Chopda

Roll No.: 150096725128

Program: B.Tech Computer Science & Engineering

Semester : II

Project Title: OS Virtual Core Task Thread Supervisor

Problem Statement No: 115

Technology: Data Structure And Algorithm in CPP

Source Code Repository: [GitHub](#)

OS Virtual Core Task Thread Supervisor

1. Problem Statement

Modern operating systems execute multiple programs simultaneously such as web browsers, media players, file downloads, system services, and background applications. Since processor cores can execute only one instruction stream at a time, the operating system must efficiently manage CPU time, resources, priorities, and dependencies among tasks.

The objective of this project is to simulate a kernel-level operating system scheduler capable of managing multiple concurrent processes while ensuring fairness, efficiency, security, and optimal resource utilization. The system demonstrates key operating system concepts including task scheduling, context switching, dependency management, resource allocation, priority scheduling, and multi-core load balancing using advanced Data Structures and Algorithms.

2. Objectives

The primary objectives of this project are:

- Design and implement a virtual operating system scheduler.
- Simulate multitasking through process scheduling and context switching.
- Manage process priorities using efficient scheduling algorithms.
- Maintain a centralized task directory for fast task lookup and management.
- Implement dependency tracking between processes and resources.
- Prevent resource conflicts through lock management.
- Distribute workloads dynamically across multiple CPU cores.
- Demonstrate practical applications of Data Structures and Algorithms in operating systems.

3. Design / Architecture

System Components

Task Directory

Stores and manages information related to all active tasks including Task ID, Process Name, Priority Level, State, Core Assignment, and CPU Usage.

Security Manager

Verifies process authorization before execution and ensures only approved tasks are allowed to run.

Scheduler Engine

Responsible for process scheduling, context switching, task execution, and priority management.

Dependency Manager

Tracks relationships between processes and resources using a graph-based structure.

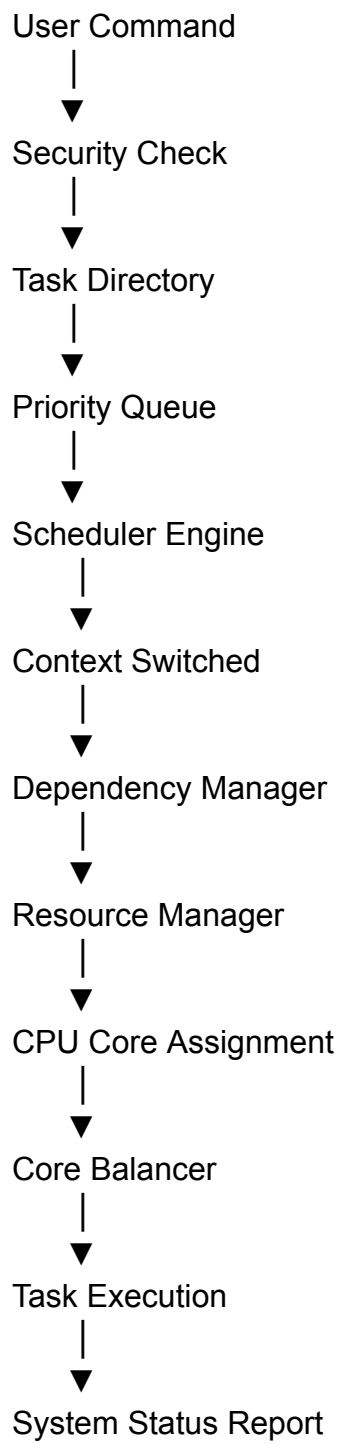
Resource Manager

Controls resource allocation and lock management to prevent conflicts among processes.

Core Balancer

Monitors processor workloads and redistributes tasks across CPU cores for balanced utilization.

System Architecture Flow



4. Algorithm Description

Round Robin Scheduling

The scheduler uses the Round Robin scheduling algorithm to provide fair CPU time allocation among processes. Each task receives a fixed time slice and is then returned to the queue if execution is not completed.

Steps

1. Add process to ready queue.
2. Assign CPU time slice.
3. Execute process.
4. Save process context.
5. Move process to end of queue if incomplete.
6. Repeat until all processes finish execution.

Priority Scheduling

Processes are classified into three levels:

- CRITICAL
- HIGH
- NORMAL

Higher-priority processes are scheduled before lower-priority processes.

Context Switching

The scheduler saves the current state of a running process before switching to another process. When the process resumes, the previously saved state is restored.

Functions Used:

- `saveContextToStack()`
- `restoreContextFromStack()`

Dependency Management

Dependencies between tasks are represented using a graph structure.

Example:

music.exe → audio.exe

This ensures that dependent tasks are handled correctly and resource conflicts are minimized.

Load Balancing Algorithm

The Core Balancer continuously monitors the workload on all processor cores.

Steps

- 1. Measure workload of each core.
- 2. Identify overloaded core.
- 3. Identify least loaded core.
- 4. Migrate tasks when imbalance exceeds threshold.
- 5. Update task assignments.

This improves CPU utilization and system efficiency.

5. Complexity Analysis

Operation	Time Complexity
Add Task	O(1)
Delete Task	O(1)

Search Task	$O(1)$
Context Save	$O(1)$
Context Restore	$O(1)$
Insert into Priority Queue	$O(\log N)$
Remove from Priority Queue	$O(\log N)$
Dependency Traversal	$O(V + E)$
Scheduler Cycle	$O(N \log N)$
Core Balancing	$O(N)$

Space Complexity

Overall Space Complexity:

$O(N)$

where N represents the number of active processes maintained within the system.

6. Screenshots of Execution

Screenshot 1 – System Initialization

```
○ nishi@NISHIs-MacBook-Air DSA-Project % make run
```

```
./bin/virtual_os
```

```
✓ system.exe STARTED
```

```
✓ Task ID: 1
```

```
✓ Assigned to Core 0
```

```
✓ Security Check PASSED
```

```
✓ memory.exe STARTED
```

```
✓ Task ID: 2
```

```
✓ Assigned to Core 1
```

```
✓ Security Check PASSED
```

```
✓ browser.exe STARTED
```

```
✓ Task ID: 3
```

```
✓ Assigned to Core 2
```

```
✓ Security Check PASSED
```

```
✓ music.exe STARTED
```

```
✓ Task ID: 4
```

```
✓ Assigned to Core 3
```

```
✓ Security Check PASSED
```

```
✓ audio.exe STARTED
```

```
✓ Task ID: 5
```

```
✓ Assigned to Core 0
```

```
✓ Security Check PASSED
```

```
✓ game.exe STARTED
```

```
✓ Task ID: 6
```

```
✓ Assigned to Core 1
```

```
✓ Security Check PASSED
```

```
✓ chrome.exe STARTED
```

```
✓ Task ID: 7
```

```
✓ Assigned to Core 2
```

```
✓ Security Check PASSED
```

```
✓ spotify.exe STARTED
```

```
✓ Task ID: 8
```

```
✓ Assigned to Core 3
```

```
✓ Security Check PASSED
```

```
✓ vscode.exe STARTED
```

```
✓ Task ID: 9
```

```
✓ Assigned to Core 0
```

```
✓ Security Check PASSED
```

```
✓ notepad.exe STARTED
```

```
✓ Task ID: 10
```

```
✓ Assigned to Core 1
```

```
✓ Security Check PASSED
```

```
✓ calculator.exe STARTED
```

```
✓ Task ID: 11
```

```
✓ Assigned to Core 2
```

```
✓ Security Check PASSED
```

```
VIRTUAL OS SUPERVISOR  
v2.0
```

```
CPU Usage : 82%
```

```
Tasks Running : 11
```

```
Tasks Waiting : 0
```

```
Core Count : 4
```

```
Commands: start, kill, pause, resume, deps, req, rel, demo, list, schedule N, queues, cores, stats, exit  
[Sun Jun 14 11:52:04 2026] OS@virtual:~$ █
```


Description:

The Virtual OS Supervisor initializes and prepares the scheduling environment for task execution.

Screenshot 2 – Task Creation and Security Verification

```
○ [Sun Jun 14 11:56:01 2026] OS@virtual:~$ start browser.exe BR012345
✓ browser.exe STARTED
✓ Task ID: 13
✓ Assigned to Core 0
✓ Security Check PASSED

[Sun Jun 14 11:56:40 2026] OS@virtual:~$ █
```

Description:

Tasks are created, assigned unique IDs, allocated processor cores, and validated through security checks.

Screenshot 3 – Demo Mode Execution

```
○ [Sun Jun 14 12:01:05 2026] OS@virtual:~$ demo

===== DEMO MODE =====
✓ browser.exe STARTED
✓ Task ID: 20
✓ Assigned to Core 0
✓ Security Check PASSED

✓ music.exe STARTED
✓ Task ID: 21
✓ Assigned to Core 2
✓ Security Check PASSED

✓ audio.exe STARTED
✓ Task ID: 22
✓ Assigned to Core 0
✓ Security Check PASSED

  ✂ PREEMPTION: CRITICAL task audio.exe paused browser.exe on Core 0
✓ game.exe STARTED
✓ Task ID: 23
✓ Assigned to Core 1
✓ Security Check PASSED

🔗 Dependency added: music.exe depends on audio.exe
  [Resource] Task audio.exe waiting for SoundCard (owned by ID 4)

--- RUNNING SCHEDULER (3 cycles) ---
```

```
===== SCHEDULER RUNNING =====  
  
--- CYCLE 1 ---  
  ✂ HIGH: system.exe  
  ✂ HIGH: memory.exe  
  ✂ HIGH: antivirus.exe  
  ✂ HIGH: audio.exe  
  ✂ HIGH: antivirus.exe  
  ✂ HIGH: audio.exe  
  ✂ HIGH: vscode.exe  
  ✂ HIGH: browser.exe  
  ✂ HIGH: browser.exe  
  ✂ HIGH: chrome.exe  
  ✂ HIGH: browser.exe  
  > NORMAL: music.exe  
  > NORMAL: game.exe  
  > NORMAL: music.exe  
  > NORMAL: game.exe  
  🔄 AUTO-BALANCE: Migrating game.exe from Core 3 to Core 0 (Diff: 600ms)
```

```
--- CYCLE 2 ---  
  ✂ HIGH: system.exe  
  ✂ HIGH: memory.exe  
  ✂ HIGH: audio.exe  
  ✂ HIGH: vscode.exe  
  ✂ HIGH: antivirus.exe  
  ✂ HIGH: antivirus.exe  
  ✂ HIGH: audio.exe  
  ✂ HIGH: browser.exe  
  ✂ HIGH: chrome.exe  
  ✂ HIGH: browser.exe  
  ✂ HIGH: browser.exe  
  > NORMAL: music.exe  
  > NORMAL: game.exe  
  > NORMAL: music.exe  
  > NORMAL: game.exe  
  🔄 AUTO-BALANCE: Migrating audio.exe from Core 0 to Core 3 (Diff: 1300ms)
```

```
--- CYCLE 3 ---  
  ✂ HIGH: system.exe  
  ✂ HIGH: memory.exe  
  ✂ HIGH: vscode.exe  
  ✂ HIGH: audio.exe  
  ✂ HIGH: antivirus.exe  
  ✂ HIGH: audio.exe  
  ✂ HIGH: antivirus.exe  
  ✂ HIGH: browser.exe  
  ✂ HIGH: browser.exe  
  ✂ HIGH: chrome.exe  
  ✂ HIGH: browser.exe  
  > NORMAL: music.exe  
  > NORMAL: game.exe  
  > NORMAL: music.exe  
  > NORMAL: game.exe  
  🔄 AUTO-BALANCE: Migrating browser.exe from Core 0 to Core 3 (Diff: 1000ms)
```

TASK DIRECTORY						
ID	SecID	Name	State	Priority	Core	CPU(ms)
1	SYS12345	system.exe	RUNNING	CRITICAL	0	1000
2	MEM12345	memory.exe	RUNNING	CRITICAL	1	1000
3	BR012345	browser.exe	RUNNING	HIGH	2	1000
4	MUS45678	music.exe	PAUSED	NORMAL	3	0
5	AUD78910	audio.exe	WAITING	CRITICAL	0	0
6	GAM11121	game.exe	PAUSED	NORMAL	1	300
7	BR012345	chrome.exe	RUNNING	HIGH	2	1000
8	MUS45678	spotify.exe	PAUSED	NORMAL	3	0
9	AUD78910	vscode.exe	RUNNING	CRITICAL	0	1000
10	GAM11121	notepad.exe	PAUSED	NORMAL	1	300
11	BR012345	calculator.exe	WAITING	HIGH	2	300
12	BR012345	browser.exe	PAUSED	NORMAL	3	0
13	BR012345	browser.exe	PAUSED	NORMAL	0	700
14	BR0123	browser.exe	RUNNING	HIGH	3	1000
15	MUS456	music.exe	RUNNING	NORMAL	1	1000
16	AUD789	audio.exe	RUNNING	CRITICAL	3	1000
17	GAM789	game.exe	RUNNING	NORMAL	0	1000
18	VIR99999	antivirus.exe	RUNNING	CRITICAL	1	700
19	VIR99999	antivirus.exe	RUNNING	CRITICAL	2	700
20	BR0123	browser.exe	RUNNING	HIGH	3	300
21	MUS456	music.exe	RUNNING	NORMAL	2	300
22	AUD789	audio.exe	RUNNING	CRITICAL	3	300
23	GAM789	game.exe	RUNNING	NORMAL	1	300

=== Core Status ===
 Core 0: 5 tasks
 Core 1: 6 tasks
 Core 2: 5 tasks
 Core 3: 7 tasks
 [Sun Jun 14 12:13:13 2026] OS@virtual:~\$ █

Description:

The scheduler demonstrates task execution and process management using simulation mode.

Screenshot 4 – Process Preemption

○ [Sun Jun 14 11:58:58 2026] OS@virtual:~\$ start antivirus.exe VIR99999 critical ✓ antivirus.exe STARTED ✓ Task ID: 19 ✓ Assigned to Core 2 ✓ Security Check PASSED [Sun Jun 14 11:59:00 2026] OS@virtual:~\$ █

Description:

Critical processes interrupt lower-priority tasks, demonstrating preemptive scheduling.

Screenshot 5 – Dependency and Resource Management

```
○ [Sun Jun 14 11:59:00 2026] OS@virtual:~$ deps 10 11
  Dependency added: notepad.exe depends on calculator.exe
[Sun Jun 14 11:59:34 2026] OS@virtual:~$ req 10 NetworkCard
  [Resource] Task notepad.exe acquired NetworkCard
[Sun Jun 14 11:59:42 2026] OS@virtual:~$ req 11 NetworkCard
  [Resource] Task calculator.exe waiting for NetworkCard (owned by ID 10)
[Sun Jun 14 11:59:52 2026] OS@virtual:~$ █
```

Description:

Dependencies between processes and resource-lock handling are displayed.

Screenshot 6 – Scheduler Running Cycles

```
○ [Sun Jun 14 11:59:52 2026] OS@virtual:~$ schedule 2

===== SCHEDULER RUNNING =====

--- CYCLE 1 ---
  < HIGH: system.exe
  < HIGH: memory.exe
  < HIGH: antivirus.exe
  < HIGH: antivirus.exe
  < HIGH: audio.exe
  < HIGH: vscode.exe
  < HIGH: browser.exe
  < HIGH: chrome.exe
  < HIGH: browser.exe
  > NORMAL: browser.exe
  > NORMAL: music.exe
  > NORMAL: game.exe
  🔄 AUTO-BALANCE: Migrating music.exe from Core 0 to Core 1 (Diff: 1100ms)

--- CYCLE 2 ---
  < HIGH: system.exe
  < HIGH: memory.exe
  < HIGH: antivirus.exe
  < HIGH: antivirus.exe
  < HIGH: audio.exe
  < HIGH: vscode.exe
  < HIGH: browser.exe
  < HIGH: browser.exe
  < HIGH: chrome.exe
  > NORMAL: browser.exe
  > NORMAL: music.exe
  > NORMAL: game.exe
[Sun Jun 14 12:00:35 2026] OS@virtual:~$ █
```

Description:

Multiple scheduling cycles are executed while maintaining process priorities and fairness.

Screenshot 7 – Core Balancing and Task Directory

```
○ [Sun Jun 14 12:13:13 2026] OS@virtual:~$ schedule 1

===== SCHEDULER RUNNING =====

--- CYCLE 1 ---
  ⚡ HIGH: system.exe
  ⚡ HIGH: memory.exe
  ⚡ HIGH: audio.exe
  ⚡ HIGH: antivirus.exe
  ⚡ HIGH: vscode.exe
  ⚡ HIGH: audio.exe
  ⚡ HIGH: antivirus.exe
  ⚡ HIGH: browser.exe
  ⚡ HIGH: chrome.exe
  ⚡ HIGH: browser.exe
  ⚡ HIGH: browser.exe
  ➤ NORMAL: music.exe
  ➤ NORMAL: game.exe
  ➤ NORMAL: music.exe
  ➤ NORMAL: game.exe
```

Unknown Command: 'list'. Help

[Sun Jun 14 12:16:52 2026] OS@virtual:~\$ list

TASK DIRECTORY						
ID	SecID	Name	State	Priority	Core	CPU(ms)
1	SYS12345	system.exe	RUNNING	CRITICAL	0	1100
2	MEM12345	memory.exe	RUNNING	CRITICAL	1	1100
3	BR012345	browser.exe	RUNNING	HIGH	2	1100
4	MUS45678	music.exe	PAUSED	NORMAL	3	0
5	AUD78910	audio.exe	WAITING	CRITICAL	0	0
6	GAM11121	game.exe	PAUSED	NORMAL	1	300
7	BR012345	chrome.exe	RUNNING	HIGH	2	1100
8	MUS45678	spotify.exe	PAUSED	NORMAL	3	0
9	AUD78910	vscode.exe	RUNNING	CRITICAL	0	1100
10	GAM11121	notepad.exe	PAUSED	NORMAL	1	300
11	BR012345	calculator.exe	WAITING	HIGH	2	300
12	BR012345	browser.exe	PAUSED	NORMAL	3	0
13	BR012345	browser.exe	PAUSED	NORMAL	0	700
14	BR0123	browser.exe	RUNNING	HIGH	3	1100
15	MUS456	music.exe	RUNNING	NORMAL	1	1100
16	AUD789	audio.exe	RUNNING	CRITICAL	3	1100
17	GAM789	game.exe	RUNNING	NORMAL	0	1100
18	VIR99999	antivirus.exe	RUNNING	CRITICAL	1	800
19	VIR99999	antivirus.exe	RUNNING	CRITICAL	2	800
20	BR0123	browser.exe	RUNNING	HIGH	3	400
21	MUS456	music.exe	RUNNING	NORMAL	2	400
22	AUD789	audio.exe	RUNNING	CRITICAL	3	400
23	GAM789	game.exe	RUNNING	NORMAL	1	400

[Sun Jun 14 12:16:56 2026] OS@virtual:~\$

Description:

The system displays task information, core assignments, and automatic load balancing across processor cores.

7. Results and Findings

The Virtual OS Supervisor v2.0 project was successfully implemented and tested to simulate the behavior of a modern operating system scheduler.

The system efficiently manages multiple concurrent tasks while demonstrating important operating system concepts such as task scheduling, context switching, dependency management, resource allocation, and multi-core load balancing.

During execution, the scheduler successfully assigned processes to available CPU cores, maintained process priorities, and performed context switching whenever higher-priority tasks required immediate execution.

Security verification checks were successfully performed before allowing processes to execute. Dependency management correctly tracked relationships between tasks and resources, while the resource-lock mechanism prevented conflicts and ensured safe execution.

The implementation of priority queues enabled critical system processes to receive CPU time before normal processes. The Round Robin scheduler ensured fairness among all active tasks and reduced the possibility of starvation.

The Core Balancer continuously monitored processor workloads and dynamically migrated tasks whenever imbalance was detected, improving overall CPU utilization.

Key observations include:

- Successful execution of concurrent processes.
- Effective context switching implementation.
- Proper dependency management.
- Accurate priority-based scheduling.
- Efficient resource allocation.
- Dynamic load balancing across multiple CPU cores.
- Stable system performance during execution cycles.

8. Conclusion

Virtual OS Supervisor v2.0 successfully demonstrates the practical implementation of operating system scheduling concepts using Data Structures and Algorithms.

The project integrates multiple data structures including Hash Maps, Priority Queues, Queues, Stacks, and Graphs to efficiently manage processes and resources. Through the use of scheduling algorithms and resource management techniques, the system achieves fairness, efficiency, and stability.

The implementation highlights the importance of process scheduling, dependency tracking, context switching, and load balancing in modern multi-core computing environments.

Overall, the project successfully fulfills all requirements specified in the problem statement and provides a comprehensive simulation of a kernel-level task scheduler.

9. GitHub Repository Link

Repository Name:

Virtual_OS

Repository URL:

https://github.com/nishhhi11/Virtual_OS

The repository contains the complete source code, implementation modules, screenshots, documentation, and supporting files required for project evaluation.